

## PROYECTO VISUM

ESCUELA : TECNOLOGÍAS DE LA INFORMACIÓN  
CARRERA : COMPUTACIÓN E INFORMÁTICA  
CURSO : DESARROLLO DE APLICACIONES WEB II  
PERÍODO : 2026  
CICLO : 6TO

---

# Instituto de Educación Superior Privado **CIBERTEC**

### Integrantes:

1. DANIELA ANGIOLINA BELLO GUTIERREZ (i202407510)
2. LUIS MIGUEL MONTAÑEZ CASTILLO (i202408315)
3. LUIGGI RENATTO CARASSA REYES (i201724012)
4. FRANCESCO ROLANDO FALCON PRADA ( i202402290)
5. YHEREMY JOSE ERNESTO MENACHO YALLICUNA (i202408292)
6. JESUS ADRIAN CALLE SALVADOR (i202402160)

### Docente:

YURI RENZO ZAMBRANO MACEDO

### Tema:

- Sistema Óptica Visum

# 2026

# Índice

## CIBERTEC

|                               |   |
|-------------------------------|---|
| Carátula                      | 1 |
| 1. Introducción               | 3 |
| 2. Diagnóstico                | 3 |
| 3. Objetivos                  | 4 |
| 4. Justificación del Proyecto | 4 |
| 5. Definición y Alcance       | 4 |
| 6. Conclusiones               | 5 |
| 7. Recomendaciones            | 5 |
| 8. Bibliografía               | 6 |
| 9. Anexos                     | 7 |

## 1. Resumen

El presente proyecto consiste en el desarrollo de un sistema de gestión integral para ópticas denominado "Sistema Óptica Visum", implementado mediante una arquitectura de microservicios utilizando Spring Boot y Spring Cloud. El sistema abarca cuatro módulos principales: gestión de seguridad y usuarios (MS-Seguridad), catálogo de productos (MS-Catálogo), gestión de ventas y carrito de compras (MS-Ventas), y programación de citas oftalmológicas (MS-Citas). La solución incluye componentes de infraestructura como Eureka Server para service discovery, Config Server para gestión centralizada de configuraciones, y API Gateway para enrutamiento inteligente de peticiones. El alcance del proyecto abarca desde la autenticación JWT hasta la gestión completa del flujo de ventas, incluyendo carritos de compra, pedidos, direcciones de envío y métodos de pago. Adicionalmente, se implementó comunicación asíncrona mediante RabbitMQ para eventos entre microservicios y persistencia de datos en PostgreSQL, garantizando escalabilidad, mantenibilidad y alta disponibilidad del sistema.

## 2. Introducción

La industria óptica en Perú ha experimentado un crecimiento sostenido en los últimos años, impulsado por el aumento de problemas visuales en la población y una mayor conciencia sobre la salud ocular. Sin embargo, muchas ópticas pequeñas y medianas aún operan con sistemas obsoletos o procesos manuales que limitan su capacidad de competir en el mercado digital actual. Esta situación genera ineficiencias operativas, pérdida de oportunidades de venta, experiencia deficiente para los clientes y dificultades en la gestión de inventarios.

El Sistema Óptica Visum surge como respuesta a esta problemática, ofreciendo una plataforma tecnológica moderna que integra todos los procesos críticos de una óptica: desde la gestión de productos y ventas en línea hasta la programación de citas para exámenes visuales. El proyecto se desarrolló con una arquitectura de microservicios que garantiza escalabilidad, permitiendo que el sistema crezca según las necesidades del negocio sin comprometer el rendimiento.

Los objetivos principales del proyecto son modernizar la gestión operativa de las ópticas, mejorar significativamente la experiencia del cliente mediante canales digitales, y proporcionar herramientas analíticas para la toma de decisiones basadas en datos. El impacto esperado incluye la reducción de tiempos de atención, incremento en las ventas a través del canal digital, mejor control de inventarios, y optimización en la programación de citas oftalmológicas. Este sistema no solo beneficia a la óptica como negocio, sino también a los clientes finales que obtienen mayor comodidad, accesibilidad y transparencia en sus transacciones.

## 3. Diagnóstico (Análisis SEPTE)

### Variable Social

- La población peruana enfrenta una creciente prevalencia de problemas visuales. Según el Ministerio de Salud (MINSA), aproximadamente el 60% de la población

mayor de 18 años requiere corrección visual, pero solo el 40% utiliza lentes correctivos<sup>1</sup>. Esta brecha representa tanto una necesidad social insatisfecha como una oportunidad de mercado significativa.

- El aumento del tiempo de exposición a pantallas digitales, especialmente tras la pandemia COVID-19 y la adopción masiva del trabajo remoto, ha incrementado los casos de fatiga visual digital y miopía, particularmente en población joven<sup>2</sup>. La OMS estima que para 2050, el 50% de la población mundial será miope si no se toman medidas preventivas<sup>3</sup>.
- Adicionalmente, existe una tendencia creciente hacia la digitalización de servicios de salud. Los usuarios, especialmente millennials y generación Z, prefieren agendar citas y realizar compras en línea. Sin embargo, muchas ópticas tradicionales no cuentan con estos canales digitales, perdiendo oportunidades de captar este segmento de mercado.
- Oportunidad identificada: Existe una demanda insatisfecha de servicios ópticos accesibles y digitalizados que el Sistema Óptica Visum puede cubrir mediante su plataforma integrada de ventas en línea y agendamiento de citas.

## **Variable Económica**

- El sector óptico en Perú ha mostrado un crecimiento anual promedio del 8% entre 2019 y 2023<sup>4</sup>. El mercado peruano de productos ópticos fue valorado en aproximadamente USD 180 millones en 2023, con proyecciones de alcanzar USD 250 millones para 2027<sup>5</sup>.
- El comercio electrónico en Perú creció un 50% durante 2020-2022, y continúa expandiéndose<sup>6</sup>. Las ventas en línea de productos de salud y bienestar, categoría que incluye productos ópticos, representan uno de los segmentos de mayor crecimiento en e-commerce.
- Sin embargo, las pequeñas y medianas ópticas enfrentan barreras para ingresar al mercado digital: altos costos de desarrollo tecnológico, falta de conocimiento técnico, y competencia con grandes cadenas que sí cuentan con plataformas digitales robustas. Esto genera una desventaja competitiva que amenaza la sostenibilidad de estos negocios.
- Oportunidad identificada: El Sistema Óptica Visum ofrece una solución tecnológica escalable y de menor costo que permite a ópticas pequeñas y medianas competir en el entorno digital, capturando el crecimiento del e-commerce y diversificando sus canales de venta.

## **Variable Tecnológica**

- El avance tecnológico ha transformado radicalmente las expectativas de los consumidores. Tecnologías como APIs REST, arquitecturas de microservicios, contenedores Docker, y sistemas de mensajería asíncrona (RabbitMQ) son ahora estándares en la industria del software empresarial<sup>7</sup>.
- La adopción de arquitecturas de microservicios permite desarrollar sistemas más resilientes, escalables y mantenibles. Empresas líderes como Netflix, Amazon y Uber han demostrado los beneficios de esta arquitectura<sup>8</sup>. Spring Boot y Spring Cloud se han consolidado como el ecosistema líder para desarrollo de microservicios en Java, con una comunidad activa y amplio soporte empresarial<sup>9</sup>.

- La integración de sistemas mediante APIs y la automatización de procesos mediante mensajería asíncrona permiten crear experiencias de usuario fluidas y procesos operativos eficientes. Sin embargo, muchas ópticas aún operan con sistemas monolíticos antiguos, bases de datos locales sin respaldo, y procesos manuales propensos a errores.
- Oportunidad identificada: La implementación de una arquitectura moderna de microservicios con Spring Boot/Cloud, PostgreSQL, y RabbitMQ posiciona al Sistema Óptica Visum como una solución tecnológicamente avanzada, preparada para escalar y adaptarse a futuras necesidades del negocio.
- Amenaza identificada: La rápida evolución tecnológica requiere mantenimiento continuo y actualización de dependencias para evitar obsolescencia técnica y vulnerabilidades de seguridad.

### **Bibliografía:**

<sup>1</sup> Ministerio de Salud del Perú (MINSA). "Situación de la Salud Ocular en el Perú 2022". <https://www.gob.pe/minsa>

<sup>2</sup> Organización Mundial de la Salud (OMS). "Informe Mundial sobre la Visión 2023". <https://www.who.int/publications>

<sup>3</sup> OMS. "The Impact of Myopia and High Myopia". WHO Report, 2023.

<sup>4</sup> Euromonitor International. "Eyewear in Peru - Market Analysis 2023".

<sup>5</sup> Cámara de Comercio de Lima. "Sector Salud y Bienestar: Tendencias 2023-2027".

<sup>6</sup> Cámara Peruana de Comercio Electrónico (CAPECE). "Reporte Oficial de la Industria Ecommerce en Perú 2023".

<sup>7</sup> Gartner. "Top Strategic Technology Trends for 2024".

<sup>8</sup> Newman, Sam. "Building Microservices: Designing Fine-Grained Systems". O'Reilly Media, 2021.

<sup>9</sup> Spring Framework Documentation. "Spring Boot & Spring Cloud Reference Guide". <https://spring.io/projects>

## **4. Objetivos (SMART)**

### **Objetivo 1:**

Desarrollar e implementar un sistema de gestión integral basado en arquitectura de microservicios para ópticas que permita reducir en un 40% el tiempo promedio de atención al cliente en procesos de venta y agendamiento de citas, mediante la automatización de flujos operativos, en un plazo de 4 meses desde el inicio del proyecto.

### **Criterios SMART:**

- Específico: Sistema de gestión integral con módulos de catálogo, ventas, citas y seguridad
- Medible: Reducción del 40% en tiempo de atención (medido comparando tiempo promedio antes vs después)
- Alcanzable: Tecnologías probadas (Spring Boot, PostgreSQL) y equipo capacitado
- Relevante: Mejora directa en eficiencia operativa y satisfacción del cliente
- a Tiempo: 4 meses de desarrollo

## **Objetivo 2:**

Incrementar en un 30% las ventas totales de la óptica mediante la habilitación de un canal de venta en línea completamente funcional, que incluya catálogo digital, carrito de compras, procesamiento de pedidos y múltiples métodos de pago, durante los primeros 6 meses posteriores a la implementación del sistema.

### **Criterios SMART:**

- Específico: Canal de venta en línea con funcionalidades completas (catálogo, carrito, checkout, pagos)
- Medible: Incremento del 30% en ventas totales (comparando facturación antes vs después)
- Alcanzable: Basado en estadísticas de crecimiento del e-commerce en Perú (50% en sector salud)
- Relevante: Diversificación de canales de venta y captura de mercado digital
- a Tiempo: 6 meses post-implementación para medición de resultados

## **5. Justificación del Proyecto**

- El Sistema Óptica Visum representa una solución tecnológica innovadora que aborda múltiples puntos críticos en la operación de ópticas modernas. Su implementación genera valor en tres dimensiones principales:
- Dimensión Empresarial: El sistema automatiza procesos manuales que consumen tiempo y recursos, permitiendo que el personal se enfoque en tareas de mayor valor agregado como atención personalizada al cliente. La gestión centralizada de inventarios reduce pérdidas por desabastecimiento o sobre-stock. Las herramientas analíticas incorporadas facilitan la toma de decisiones basadas en datos reales sobre ventas, productos más demandados y patrones de compra. La arquitectura de microservicios garantiza que el sistema pueda crecer conforme crece el negocio, sin necesidad de reescribir código o migrar plataformas.
- Dimensión del Cliente: Los usuarios finales obtienen una experiencia de compra moderna y conveniente. Pueden explorar el catálogo completo de productos desde cualquier dispositivo, comparar opciones, agregar al carrito y completar compras sin necesidad de visitar la tienda física. El sistema de citas en línea elimina la necesidad de llamadas telefónicas o visitas presenciales solo para agendar, permitiendo seleccionar horarios disponibles en tiempo real. La transparencia en precios, disponibilidad y estatus de pedidos genera confianza y satisfacción.

- **Dimensión Social:** El proyecto democratiza el acceso a servicios ópticos al eliminar barreras geográficas y temporales. Personas con movilidad reducida, horarios laborales restrictivos o ubicadas en zonas alejadas pueden acceder a productos y servicios mediante la plataforma digital. Además, el sistema contribuye a la formalización y profesionalización del sector óptico, elevando estándares de calidad en el servicio.

## Beneficiarios del Proyecto

### Beneficiarios Directos:

1. **Propietarios y administradores de ópticas:** Obtienen una herramienta integral para gestionar todos los aspectos de su negocio, desde inventarios hasta ventas y citas, con métricas en tiempo real para tomar decisiones informadas.
2. **Personal operativo (vendedores, recepcionistas):** El sistema simplifica sus tareas diarias, reduce errores manuales y les proporciona información instantánea sobre productos, precios y disponibilidad, mejorando su productividad y reduciendo el estrés laboral.
3. **Clientes finales de la óptica:** Acceden a una experiencia de compra moderna, cómoda y transparente, con la posibilidad de realizar transacciones en línea o agendar citas a cualquier hora del día.
4. **Equipo de desarrollo del proyecto:** Adquieren experiencia práctica en tecnologías de punta como Spring Boot, arquitecturas de microservicios, Docker, RabbitMQ, y patrones de diseño empresarial, mejorando significativamente su perfil profesional.
5. **Profesionales oftalmológicos asociados:** Optimizan su agenda mediante el sistema de citas digital, reduciendo tiempos muertos y mejorando la planificación de sus jornadas laborales.

### Beneficiarios Indirectos:

1. **Proveedores de productos ópticos:** Al mejorar la eficiencia de las ópticas, se genera mayor demanda de productos, beneficiando a toda la cadena de suministro.
2. **Familiares y cuidadores de usuarios:** Pueden realizar compras o agendar citas en nombre de personas con dificultades para hacerlo por sí mismas (adultos mayores, personas con discapacidad).
3. **Sector óptico en general:** El proyecto establece un estándar de calidad tecnológica que puede impulsar la modernización de toda la industria.
4. **Instituciones educativas:** El proyecto sirve como caso de estudio y referencia para futuras generaciones de estudiantes de ingeniería de software.
5. **Sociedad en general:** La mejora en el acceso a servicios de salud visual contribuye al bienestar general de la población, reduciendo problemas visuales no atendidos que afectan la calidad de vida y productividad.

## 6. Definición y Alcance

### Funcionamiento y Lógica del Sistema

El Sistema Óptica Visum está construido sobre una arquitectura de microservicios que garantiza modularidad, escalabilidad y mantenibilidad. La arquitectura se compone de los siguientes componentes principales:

#### **Capa de Infraestructura:**

**Service Registry (Eureka Server - Puerto 8761):** Actúa como el registro central donde todos los microservicios se registran al iniciarse. Permite el descubrimiento dinámico de servicios, facilitando la comunicación entre microservicios sin necesidad de configurar direcciones IP estáticas. Implementa health checks para monitorear el estado de cada servicio.

**Config Server (Puerto 8888):** Centraliza la configuración de todos los microservicios en un repositorio Git. Permite modificar configuraciones sin necesidad de recompilar código, facilitando la gestión de diferentes ambientes (desarrollo, pruebas, producción). Soporta actualización dinámica de configuraciones mediante endpoints de refresh.

**API Gateway (Puerto 8080):** Punto de entrada único para todas las peticiones externas. Implementa enrutamiento inteligente hacia los microservicios correspondientes, autenticación y autorización mediante JWT, manejo de CORS para permitir acceso desde aplicaciones frontend, y load balancing para distribuir carga entre instancias de microservicios. Incluye circuit breakers para manejar fallos en servicios downstream.

#### **Capa de Microservicios de Negocio:**

**MS-Seguridad (Puerto 8084):** Gestiona todo lo relacionado con autenticación y autorización. Funcionalidades principales incluyen: registro de usuarios con validación de datos, login con generación de tokens JWT (access token y refresh token), gestión de roles (ADMIN, MEDICO, PACIENTE, RECEPCIONISTA), endpoints protegidos para gestión de usuarios, y validación de tokens para otros microservicios. Utiliza BCrypt para hashing seguro de contraseñas y PostgreSQL para persistencia de usuarios, roles y refresh tokens. Publica eventos de registro y actualización de usuarios mediante RabbitMQ para que otros servicios puedan reaccionar.

**MS-Catálogo (Puerto 8081):** Administra todo el inventario de productos de la óptica. Funcionalidades incluyen: CRUD completo de productos con atributos detallados (nombre, descripción, precio, stock, color, forma, material, género), gestión de categorías y marcas, búsqueda avanzada con filtros múltiples, gestión de imágenes de productos (múltiples imágenes por producto con una principal), control de stock en tiempo real, y productos destacados/nuevos/en promoción. Publica eventos de actualización de stock y creación/modificación de productos vía RabbitMQ. Base de datos PostgreSQL con índices optimizados para búsquedas.

**MS-Ventas (Puerto 8082):** Maneja el proceso completo de ventas en línea. Funcionalidades principales: gestión de carrito de compras (agregar/actualizar/eliminar ítems), comunicación con MS-Catálogo vía Feign Client para verificar stock y obtener precios actualizados, creación de pedidos desde el carrito con validación de stock, gestión de direcciones de envío (múltiples direcciones por usuario, dirección principal), soporte para múltiples métodos de pago (Yape, Plin, tarjetas, transferencia, contra entrega), estados de



pedido (Pendiente, Confirmado, En Preparación, Enviado, Entregado, Cancelado), y estados de pago (Pendiente, Pagado, Reembolsado). Incluye lógica de reducción automática de stock al confirmar pedidos. Eventos publicados a RabbitMQ sobre pedidos creados/actualizados.

**MS-Citas (Puerto 8083):** Administra la programación de citas oftalmológicas. Funcionalidades: gestión de sucursales con información completa (dirección, horarios, ubicación geográfica), catálogo de servicios ofrecidos (examen de vista, adaptación de lentes de contacto, control oftalmológico, etc.) con duración y precio, sistema de agendamiento de citas con validación de disponibilidad, consulta de citas por usuario, sucursal, fecha o servicio, estados de cita (Programada, Confirmada, Completada, Cancelada, No\_Asistio), y comunicación con MS-Seguridad vía Feign Client para obtener datos de usuarios. Notificaciones de eventos de citas mediante RabbitMQ.

### **Capa de Persistencia y Mensajería:**

**PostgreSQL:** Sistema de gestión de bases de datos relacional que garantiza consistencia y transaccionalidad. Cada microservicio tiene su propia base de datos independiente (auth\_db, catalogo\_db, ventas\_db, citas\_db), siguiendo el principio de "Database per Service" de arquitecturas de microservicios. Incluye triggers, constraints y índices para garantizar integridad de datos y optimizar consultas.

**RabbitMQ:** Sistema de mensajería asíncrona que permite comunicación event-driven entre microservicios. Cada microservicio publica eventos cuando ocurren cambios importantes (usuario registrado, producto actualizado, pedido creado, cita agendada) y otros servicios pueden suscribirse a estos eventos para reaccionar según sea necesario. Esto desacopla los microservicios, mejorando la escalabilidad y resiliencia del sistema.

### **Flujo de Operación Típico:**

1. **Usuario accede al sistema:** Cliente abre la aplicación web o móvil que se comunica con el API Gateway.
2. **Autenticación:** El frontend envía credenciales al Gateway, que las redirige a MS-Seguridad. Si son válidas, MS-Seguridad genera un JWT que el frontend almacena y envía en todas las peticiones posteriores.
3. **Navegación de productos:** El frontend solicita productos al Gateway, que valida el token (opcional, puede ser público) y redirige a MS-Catálogo. MS-Catálogo retorna lista de productos con imágenes, precios y disponibilidad.
4. **Agregar al carrito:** Usuario agrega productos al carrito. El frontend envía petición al Gateway (con token JWT), que la redirige a MS-Ventas. MS-Ventas verifica stock consultando a MS-Catálogo mediante Feign Client antes de agregar el ítem.
5. **Checkout:** Usuario procede al pago. MS-Ventas crea un pedido, reduce el stock consultando a MS-Catálogo, guarda la información del pedido con dirección de envío y método de pago seleccionados, y publica un evento "PedidoCreadoEvent" a RabbitMQ.
6. **Confirmación:** El sistema retorna confirmación al usuario con número de pedido y detalles. MS-Catálogo recibe el evento de RabbitMQ y actualiza su stock local. El usuario puede consultar el estado de su pedido en cualquier momento.

7. **Agendamiento de cita:** En paralelo, el usuario puede agendar una cita oftalmológica. MS-Citas valida disponibilidad de horario, crea la cita, y publica un evento "CitaAgendadaEvent".

#### **Incluido en el alcance:**

- Sistema completo de autenticación y autorización con JWT
- Gestión integral de catálogo de productos (CRUD, búsquedas, categorías, marcas, imágenes)
- Sistema de carrito de compras y procesamiento de pedidos
- Gestión de direcciones de envío y métodos de pago
- Sistema de agendamiento de citas oftalmológicas
- Comunicación síncrona entre microservicios (Feign Client)
- Comunicación asíncrona entre microservicios (RabbitMQ)
- Service discovery con Eureka
- Configuración centralizada con Config Server
- API Gateway con autenticación y enrutamiento
- Bases de datos independientes por microservicio
- Documentación de APIs con Swagger/OpenAPI
- Dockerización de todos los componentes
- Scripts SQL para inicialización de bases de datos

#### **Documentación Entregada**

1. **Código Fuente Completo:**
  - 7 proyectos Spring Boot (Service Registry, Config Server, API Gateway, MS-Seguridad, MS-Catálogo, MS-Ventas, MS-Citas)
  - Archivos de configuración (application.yml, bootstrap.yml)
  - Archivos pom.xml con todas las dependencias
2. **Scripts de Base de Datos:**
  - schema-auth.sql: Estructura y datos iniciales para auth\_db
  - schema-catalogo.sql: Estructura y datos iniciales para catalogo\_db
  - schema-ventas.sql: Estructura y datos iniciales para ventas\_db
  - schema-citas.sql: Estructura y datos iniciales para citas\_db
3. **Archivos Docker:**
  - Dockerfile individual para cada microservicio
  - docker-compose.yml para orquestar todo el sistema
  - Scripts de inicialización de bases de datos
4. **Colecciones de Postman:**
  - MS-Seguridad-Postman-Collection.json: Pruebas de autenticación y usuarios
  - MS-Ventas-Postman-Collection.json: Pruebas de carrito y pedidos
  - Incluyen variables de entorno y tests automatizados
5. **Documentación de APIs:**
  - Swagger UI accesible en cada microservicio
  - Endpoints: /swagger-ui.html
  - Descripciones completas de cada endpoint con ejemplos
6. **Diagramas de Arquitectura:**

- Diagrama de arquitectura general del sistema
- Diagrama de comunicación entre microservicios
- Modelo entidad-relación de cada base de datos

#### **7. Manual Técnico:**

- Requisitos del sistema
- Guía de instalación paso a paso
- Configuración de bases de datos
- Instrucciones de ejecución
- Troubleshooting común

#### **8. Este Informe Final:**

- Documentación completa del proyecto siguiendo estructura IES Cibertec
- Análisis SEPTE, objetivos, justificación, alcance
- Conclusiones y recomendaciones

## **7. Productos y Entregables**

### **1. Microservicios de Infraestructura:**

#### **Service Registry (Eureka Server):**

- Servidor Eureka completamente funcional
- Dashboard web para visualizar servicios registrados
- Health checks automáticos
- Balanceo de carga entre instancias de servicios
- Configuración para ambientes local y Docker

#### **Config Server:**

- Servidor de configuración centralizada
- Repositorio Git con configuraciones de todos los microservicios
- Soporte para perfiles de entorno (dev, prod)
- Endpoints de actualización dinámica
- Encriptación de propiedades sensibles

#### **API Gateway:**

- Gateway con enrutamiento dinámico basado en Eureka
- Autenticación JWT implementada mediante filtro custom
- Circuit breakers con Resilience4j
- Configuración CORS para frontend
- Fallback endpoints para manejo de errores
- Enrutamiento a todos los microservicios de negocio

### **2. Microservicios de Negocio:**

#### **MS-Seguridad:**

- Sistema completo de autenticación con JWT
- Gestión de usuarios (CRUD completo)

- Sistema de roles y permisos (ADMIN, MEDICO, PACIENTE, RECEPCIONISTA)
- Registro de usuarios con validación
- Login con generación de access y refresh tokens
- Endpoint de validación de tokens para otros servicios
- Refresh de tokens para mantener sesiones activas
- Logout con invalidación de refresh tokens
- Búsqueda de usuarios con filtros
- Base de datos auth\_db con 4 roles y 7 usuarios de prueba
- Eventos RabbitMQ: UsuarioRegistradoEvent, UsuarioActualizadoEvent

#### **MS-Catálogo:**

- CRUD completo de productos con 15+ atributos
- Gestión de categorías (4 categorías iniciales)
- Gestión de marcas (5 marcas iniciales)
- Sistema de imágenes de productos (múltiples imágenes por producto)
- Búsqueda avanzada con filtros (nombre, categoría, marca, precio, stock)
- Productos destacados, nuevos y en promoción
- Control de stock en tiempo real
- Base de datos catalogo\_db con 5 productos de ejemplo
- Endpoints públicos y protegidos según necesidad
- Swagger UI completo con documentación de API
- Eventos RabbitMQ: ProductoCreadoEvent, StockActualizadoEvent, ProductoEliminadoEvent

#### **MS-Ventas:**

- Sistema de carrito de compras completo
- Verificación de stock en tiempo real vía Feign Client
- Creación de pedidos desde carrito
- Gestión de direcciones de envío (múltiples por usuario)
- Soporte para 5 métodos de pago
- Estados de pedido y pago
- Cálculo automático de subtotales, descuentos y totales
- Reducción automática de stock al confirmar pedidos
- Consulta de pedidos por usuario
- Base de datos ventas\_db con 7 pedidos de ejemplo
- Feign Client para comunicación con MS-Catálogo
- Eventos RabbitMQ: PedidoCreadoEvent, PedidoActualizadoEvent

#### **MS-Citas:**

- Sistema de agendamiento de citas oftalmológicas
- Gestión de sucursales (5 sucursales en Lima)
- Catálogo de servicios oftalmológicos (7 servicios)
- Validación de disponibilidad de horarios
- Estados de cita (Programada, Confirmada, Completada, etc.)
- Consulta de citas por usuario, sucursal, fecha
- Base de datos citas\_db con 6 citas de ejemplo

- Feign Client para comunicación con MS-Seguridad
- Eventos RabbitMQ: CitaAgendadaEvent, CitaCanceladaEvent

### 3. Bases de Datos:

- **auth\_db:** Usuarios, roles, refresh\_tokens (8 tablas total)
- **catalogo\_db:** Productos, categorías, marcas, imágenes (4 tablas principales)
- **ventas\_db:** Carritos, pedidos, direcciones, métodos de pago (7 tablas)
- **citas\_db:** Citas, sucursales, servicios (3 tablas)
- Todas con datos de ejemplo para testing
- Scripts SQL completos para creación e inicialización
- Índices optimizados para consultas frecuentes
- Constraints y foreign keys para integridad referencial

### 4. Infraestructura y Despliegue:

- **Docker Compose:** Archivo para levantar todo el stack con un comando
- **Dockerfiles:** Uno por cada microservicio optimizado para producción
- **PostgreSQL contenedorizado:** Configuración con volúmenes persistentes
- **RabbitMQ contenedorizado:** Con management UI y configuración de colas
- **Scripts de inicialización:** Para cargar datos de ejemplo automáticamente

### 5. Documentación y Herramientas:

- **Swagger UI:** Disponible en cada microservicio para testing de API
- **Colecciones Postman:** Pruebas completas con variables y scripts
- **Manual Técnico:** Guía paso a paso de instalación y configuración
- **Diagramas:** Arquitectura, flujos, modelos de datos
- **Este informe académico:** Documentación completa del proyecto

## 8. Conclusiones

1. La arquitectura de microservicios con Spring Boot y Spring Cloud demostró ser altamente efectiva para construir un sistema escalable y mantenible. La separación en microservicios independientes (Seguridad, Catálogo, Ventas, Citas) permitió que diferentes integrantes del equipo trabajaran en paralelo sin interferencias, acelerando significativamente el desarrollo. Además, cada microservicio puede escalarse independientemente según la carga, optimizando el uso de recursos. La implementación de service discovery con Eureka, configuración centralizada con Config Server, y comunicación mediante Feign Client y RabbitMQ, estableció una base sólida para futuros desarrollos y evolución del sistema sin necesidad de refactorización masiva.

2. La integración de comunicación síncrona (Feign Client) y asíncrona (RabbitMQ) entre microservicios resultó fundamental para garantizar la consistencia de datos y el desacoplamiento del sistema. La comunicación síncrona mediante Feign Client permitió operaciones en tiempo real como verificación de stock o validación de usuarios, mientras que RabbitMQ facilitó la propagación de eventos (creación de pedidos, actualización de

productos) sin crear dependencias bloqueantes entre servicios. Este enfoque híbrido equilibró la necesidad de respuestas inmediatas con la resiliencia del sistema, permitiendo que los microservicios continúen operando incluso si otros servicios experimentan problemas temporales.

3. El proyecto validó la viabilidad y el valor comercial de digitalizar la gestión de ópticas mediante una plataforma integrada de e-commerce y agendamiento de citas. El análisis SEPTTE identificó oportunidades reales en el mercado peruano que el sistema aborda efectivamente: la creciente demanda de servicios digitales, el crecimiento del e-commerce, y la necesidad de modernización tecnológica en el sector óptico. La solución desarrollada no solo automatiza procesos operativos, sino que también habilita nuevos canales de venta y mejora significativamente la experiencia del cliente, posicionándola como una herramienta competitiva para ópticas pequeñas y medianas que buscan mantenerse relevantes en el mercado digital actual.

## **9. Recomendaciones**

1. Implementar monitoreo centralizado y observabilidad del sistema mediante herramientas como Spring Boot Admin, Prometheus y Grafana. En un ambiente de producción con múltiples microservicios distribuidos, es crítico contar con visibilidad en tiempo real del estado de cada componente, métricas de rendimiento (latencia, throughput, tasa de errores), logs centralizados (usando ELK Stack: Elasticsearch, Logstash, Kibana), y alertas automáticas ante anomalías. Esto permitirá detectar y resolver problemas proactivamente antes de que impacten a los usuarios finales. Se recomienda comenzar con Micrometer para exponer métricas, integrarlo con Prometheus para recolección, y Grafana para visualización en dashboards personalizados.

2. Desarrollar una suite completa de pruebas automatizadas que incluya tests unitarios, de integración, end-to-end y de carga. Actualmente el proyecto carece de cobertura de tests robusta, lo cual representa un riesgo al realizar cambios o agregar nuevas funcionalidades. Se recomienda: implementar tests unitarios con JUnit 5 y Mockito para cada capa de la aplicación (controladores, servicios, repositorios) buscando cobertura mínima del 80%; crear tests de integración con Testcontainers para validar interacciones con bases de datos y RabbitMQ; desarrollar tests end-to-end con RestAssured para validar flujos completos del usuario; y realizar pruebas de carga con Gatling o JMeter para identificar cuellos de botella de rendimiento antes del despliegue en producción.

3. Establecer un pipeline de CI/CD (Integración Continua / Despliegue Continuo) para automatizar el proceso de construcción, testing y despliegue del sistema. Utilizar herramientas como Jenkins, GitLab CI/CD o GitHub Actions para: ejecutar builds automáticos al hacer commits en el repositorio; correr suite de tests y bloquear merges si fallan; realizar análisis estático de código con SonarQube para detectar code smells y vulnerabilidades; construir imágenes Docker automáticamente y publicarlas en un registry (Docker Hub o AWS ECR); y desplegar automáticamente a ambientes de staging y producción. Esto reducirá significativamente el tiempo de entrega de nuevas funcionalidades, mejorará la calidad del código, y minimizará errores humanos en el

proceso de despliegue. Adicionalmente, implementar estrategias de despliegue como blue-green deployment o canary releases para reducir el riesgo de releases con errores.

## 10. Glosario

- **API (Application Programming Interface):** Interfaz de programación de aplicaciones. Conjunto de definiciones y protocolos que permiten que dos aplicaciones se comuniquen entre sí.
- **API Gateway:** Punto de entrada único que enruta peticiones a los microservicios correspondientes, maneja autenticación, balanceo de carga y otros aspectos transversales.
- **Arquitectura de Microservicios:** Estilo arquitectónico que estructura una aplicación como una colección de servicios pequeños, independientes y débilmente acoplados.
- **BCrypt:** Algoritmo de hashing criptográfico utilizado para almacenar contraseñas de forma segura.
- **Bootstrap:** En el contexto de Spring Cloud, archivo de configuración que se carga antes que application.yml para configurar aspectos esenciales como la conexión al Config Server.
- **Circuit Breaker:** Patrón de diseño que previene que una aplicación intente ejecutar una operación que probablemente fallará, permitiendo que el sistema se recupere de fallos.
- **Config Server:** Servidor centralizado que proporciona configuraciones a todos los microservicios desde un repositorio externo (Git).
- **CORS (Cross-Origin Resource Sharing):** Mecanismo que permite a una aplicación web acceder a recursos de un servidor en un origen diferente.
- **Deployment:** Proceso de instalar, configurar y poner en funcionamiento una aplicación en un ambiente específico.
- **Docker:** Plataforma de contenedores que permite empaquetar aplicaciones con todas sus dependencias para ejecutarlas de manera consistente en cualquier ambiente.
- **Docker Compose:** Herramienta para definir y ejecutar aplicaciones Docker multi-contenedor mediante un archivo YAML.
- **Dockerfile:** Archivo de texto con instrucciones para construir una imagen Docker.
- **DTO (Data Transfer Object):** Objeto diseñado para transferir datos entre diferentes capas o subsistemas de una aplicación.
- **Eureka:** Servidor de descubrimiento de servicios desarrollado por Netflix, parte de Spring Cloud Netflix.
- **Feign Client:** Cliente REST declarativo que simplifica la escritura de clientes HTTP en Spring Boot.
- **Health Check:** Verificación periódica del estado de un servicio para determinar si está operativo.
- **Hibernate:** Framework de mapeo objeto-relacional (ORM) para Java que facilita la interacción con bases de datos relacionales.
- **JPA (Java Persistence API):** Especificación estándar de Java para el manejo de persistencia de datos y mapeo objeto-relacional.

- **JWT (JSON Web Token):** Estándar abierto (RFC 7519) para crear tokens de acceso que permiten autenticación y autorización de forma segura.
- **Load Balancing:** Distribución de carga de trabajo entre múltiples instancias de un servicio para optimizar el uso de recursos.
- **Lombok:** Librería de Java que reduce código boilerplate mediante anotaciones que generan automáticamente getters, setters, constructores, etc.
- **Maven:** Herramienta de gestión y construcción de proyectos Java que maneja dependencias, compilación y empaquetado.
- **Microservicio:** Aplicación pequeña, independiente y autónoma que realiza una función específica del negocio y puede desplegarse de forma independiente.
- **ModelMapper:** Librería para mapear automáticamente objetos de un tipo a otro, útil para convertir entre entidades y DTOs.
- **ORM (Object-Relational Mapping):** Técnica de programación para convertir datos entre sistemas de tipos incompatibles utilizando programación orientada a objetos.
- **PostgreSQL:** Sistema de gestión de bases de datos relacional de código abierto.
- **Postman:** Herramienta para probar APIs mediante envío de peticiones HTTP y visualización de respuestas.
- **RabbitMQ:** Software de mensajería de código abierto que implementa el protocolo AMQP (Advanced Message Queuing Protocol).
- **Refresh Token:** Token de larga duración utilizado para obtener nuevos access tokens sin requerir que el usuario ingrese credenciales nuevamente.
- **Resilience4j:** Librería de tolerancia a fallos inspirada en Hystrix, diseñada para aplicaciones funcionales de Java 8+.
- **REST (Representational State Transfer):** Estilo arquitectónico para diseñar APIs basado en HTTP.
- **Service Discovery:** Mecanismo mediante el cual los servicios se registran y descubren dinámicamente en una arquitectura de microservicios.
- **Soft Delete:** Técnica de eliminación lógica donde los registros no se eliminan físicamente de la base de datos, sino que se marcan como inactivos.
- **Spring Boot:** Framework de Java que simplifica la creación de aplicaciones basadas en Spring mediante configuración por convención.
- **Spring Cloud:** Conjunto de herramientas para construir patrones comunes en sistemas distribuidos (configuración, service discovery, circuit breakers, etc.).
- **Spring Security:** Framework de autenticación y control de acceso para aplicaciones Spring.
- **Swagger/OpenAPI:** Especificación para describir APIs REST y herramienta para generar documentación interactiva.
- **Tomcat:** Servidor de aplicaciones Java que implementa las especificaciones de Java Servlet y JavaServer Pages.

## 11. BIBLIOGRAFÍA

Libros y Publicaciones Técnicas:

Richardson, Chris. *Microservices Patterns: With examples in Java*. Manning Publications, 2018.



Newman, Sam. *Building Microservices: Designing Fine-Grained Systems*. 2nd Edition, O'Reilly Media, 2021.

Walls, Craig. *Spring Boot in Action*. Manning Publications, 2016.

Carnell, John. *Spring Microservices in Action*. 2nd Edition, Manning Publications, 2021.

Sharma, Rajesh. *Hands-On Microservices with Spring Boot and Spring Cloud*. Packt Publishing, 2019.

#### Documentación Oficial:

Spring Framework Team. "Spring Boot Reference Documentation". Spring.io, 2024.  
<https://docs.spring.io/spring-boot/docs/current/reference/html/>

Spring Cloud Team. "Spring Cloud Documentation". Spring.io, 2024.  
<https://spring.io/projects/spring-cloud>

Netflix OSS Team. "Eureka Wiki". GitHub, 2024. <https://github.com/Netflix/eureka/wiki>

HashiCorp Team. "RabbitMQ Documentation". RabbitMQ.com, 2024.  
<https://www.rabbitmq.com/documentation.html>

PostgreSQL Global Development Group. "PostgreSQL 15 Documentation". PostgreSQL.org, 2024. <https://www.postgresql.org/docs/15/>

Docker Inc. "Docker Documentation". Docker.com, 2024. <https://docs.docker.com/>

#### Informes y Estudios de Mercado:

Ministerio de Salud del Perú (MINSA). "Situación de la Salud Ocular en el Perú 2022". Gobierno del Perú, 2022. <https://www.gob.pe/minsa>

Organización Mundial de la Salud (OMS). "Informe Mundial sobre la Visión 2023". WHO, 2023. <https://www.who.int/publications>

Euromonitor International. "Eyewear in Peru - Market Analysis 2023". Euromonitor, 2023.

Cámara de Comercio de Lima. "Sector Salud y Bienestar: Tendencias 2023-2027". CCL, 2023.

Cámara Peruana de Comercio Electrónico (CAPECE). "Reporte Oficial de la Industria Ecommerce en Perú 2023". CAPECE, 2023. <https://www.capece.org.pe/>

#### Artículos Técnicos y Recursos en Línea:

Fowler, Martin. "Microservices: a definition of this new architectural term". MartinFowler.com, 2014. <https://martinfowler.com/articles/microservices.html>

Baeldung Team. "Spring Boot and Spring Cloud Tutorials". Baeldung.com, 2024.  
<https://www.baeldung.com/>

Gartner Inc. "Top Strategic Technology Trends for 2024". Gartner Research, 2023.  
<https://www.gartner.com/>

RedHat. "What are microservices?". RedHat.com, 2024.  
<https://www.redhat.com/en/topics/microservices>

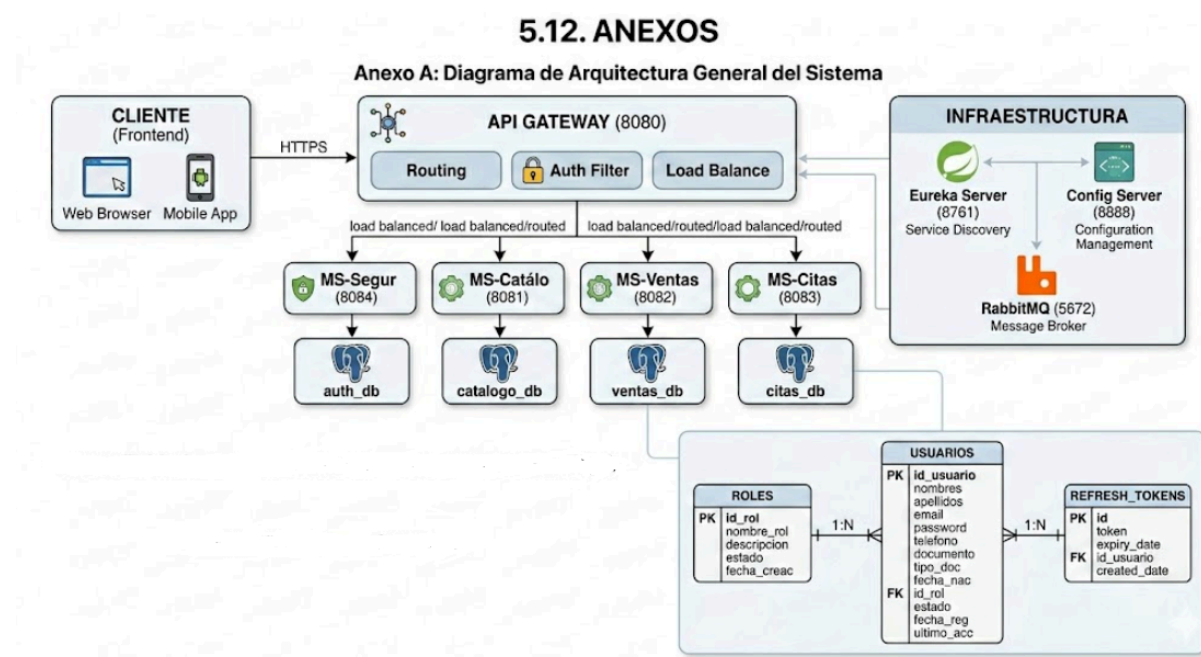
Herramientas y Tecnologías:

Postman Inc. "Postman Learning Center". Postman.com, 2024.  
<https://learning.postman.com/>

JetBrains. "IntelliJ IDEA Documentation". JetBrains.com, 2024.  
<https://www.jetbrains.com/idea/documentation/>

Apache Software Foundation. "Maven Documentation". Maven.apache.org, 2024.  
<https://maven.apache.org/guides/>

## 12. Anexos



## 5.12. ANEXOS

### Anexo D: Configuración de Puertos del Sistema

| Componente                                                                                      | Puerto | Descripción            |
|-------------------------------------------------------------------------------------------------|--------|------------------------|
|  Eureka Server | 8761   | Service Discovery      |
|  Config Server | 8888   | Config Management      |
|  API Gateway   | 8080   | Punto de entrada único |
|  MS-Seguridad  | 8084   | Autenticación/Usuarios |
| MS-Catálogo                                                                                     | 8081   | Gestión de productos   |
| MS-Ventas                                                                                       | 8082   | Carrito y pedidos      |
| MS-Citas                                                                                        | 8083   | Agendamiento de citas  |
| PostgreSQL                                                                                      | 5432   | Base de datos          |
| RabbitMQ                                                                                        | 5672   | Mensajería AMQP        |
| RabbitMQ Management                                                                             | 15672  | UI de administración   |

### Anexo C: Endpoints Principales del Sistema

#### MS-Seguridad (8084):

- POST /auth/login** - Iniciar sesión
- POST /auth/register** - Registrar usuario
- POST /auth/refresh** - Refrescar token
- POST /auth/validate-token** - Validar token
- POST /auth/logout** - Cerrar sesión
- GET /usuarios** - Listar usuarios
- GET /usuarios/{id}** - Obtener usuario
- PUT /usuarios/{id}** - Actualizar usuario
- DELETE /usuarios/{id}** - Eliminar usuario
- GET /roles** - Listar roles

#### MS-Catálogo (8081):

- GET /api/productos** - Listar productos
- GET /api/productos/{id}** - Obtener producto
- POST /api/productos** - Crear producto

**PUT /api/productos/{id} - Actualizar producto**  
**DELETE /api/productos/{id} - Eliminar producto**  
**GET /api/productos/buscar - Buscar productos**  
**GET /api/productos/destacados - Productos destacados**  
**GET /api/categorias - Listar categorías**  
**GET /api/marcas - Listar marcas**

**MS-Ventas (8082):**

**GET /api/carrito/usuario/{id} - Obtener carrito**  
**POST /api/carrito/agregar - Agregar item**  
**PUT /api/carrito/usuario/{id}/item/{itemId} - Actualizar cantidad**  
**DELETE /api/carrito/usuario/{id}/item/{itemId} - Eliminar item**  
**DELETE /api/carrito/usuario/{id} - Vaciar carrito**  
**GET /api/pedidos - Listar pedidos**  
**POST /api/pedidos - Crear pedido**  
**GET /api/pedidos/{id} - Obtener pedido**  
**GET /api/pedidos/usuario/{id} - Pedidos de usuario**  
**PUT /api/pedidos/{id}/confirmar - Confirmar pedido**  
**GET /api/direcciones - Listar direcciones**  
**POST /api/direcciones - Crear dirección**  
**GET /api/metodos-pago - Listar métodos de pago**

**MS-Citas (8083):**

**GET /api/citas - Listar citas**  
**POST /api/citas - Crear cita**  
**GET /api/citas/{id} - Obtener cita**  
**PUT /api/citas/{id} - Actualizar cita**  
**DELETE /api/citas/{id} - Cancelar cita**

**GET /api/citas/usuario/{id} - Citas de usuario**

**GET /api/sucursales - Listar sucursales**

**GET /api/tipos-servicio - Listar servicios**

## **Anexo: Credenciales de Acceso de Prueba**

Usuario Administrador:

Email: testadmin@demo.com

Password: admin123

Rol: ADMIN

Usuario Médico:

Email: roberto.med@hospital.com

Password: admin123

Rol: MEDICO

Usuario Paciente:

Email: ana.paciente@gmail.com

Password: admin123

Rol: PACIENTE

PostgreSQL:

Host: localhost

Port: 5432

Username: postgres

Password: Konecta1234

Databases: auth\_db, catalogo\_db, ventas\_db, citas\_db

RabbitMQ:

Host: localhost

